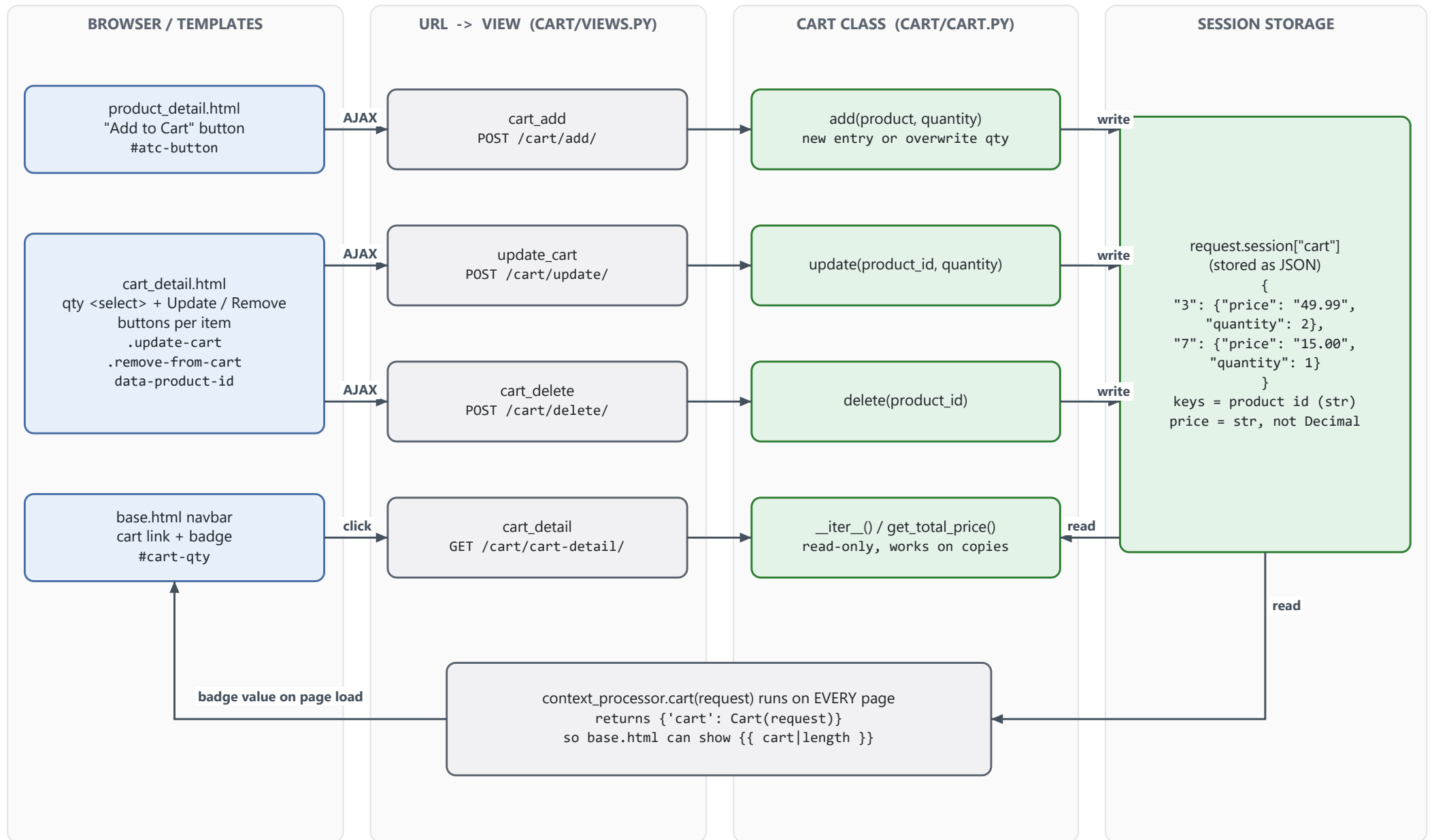


Cart overview - who talks to whom

Every cart action is an AJAX POST from a template to a view; the view uses the Cart class, which stores everything in request.session["cart"].

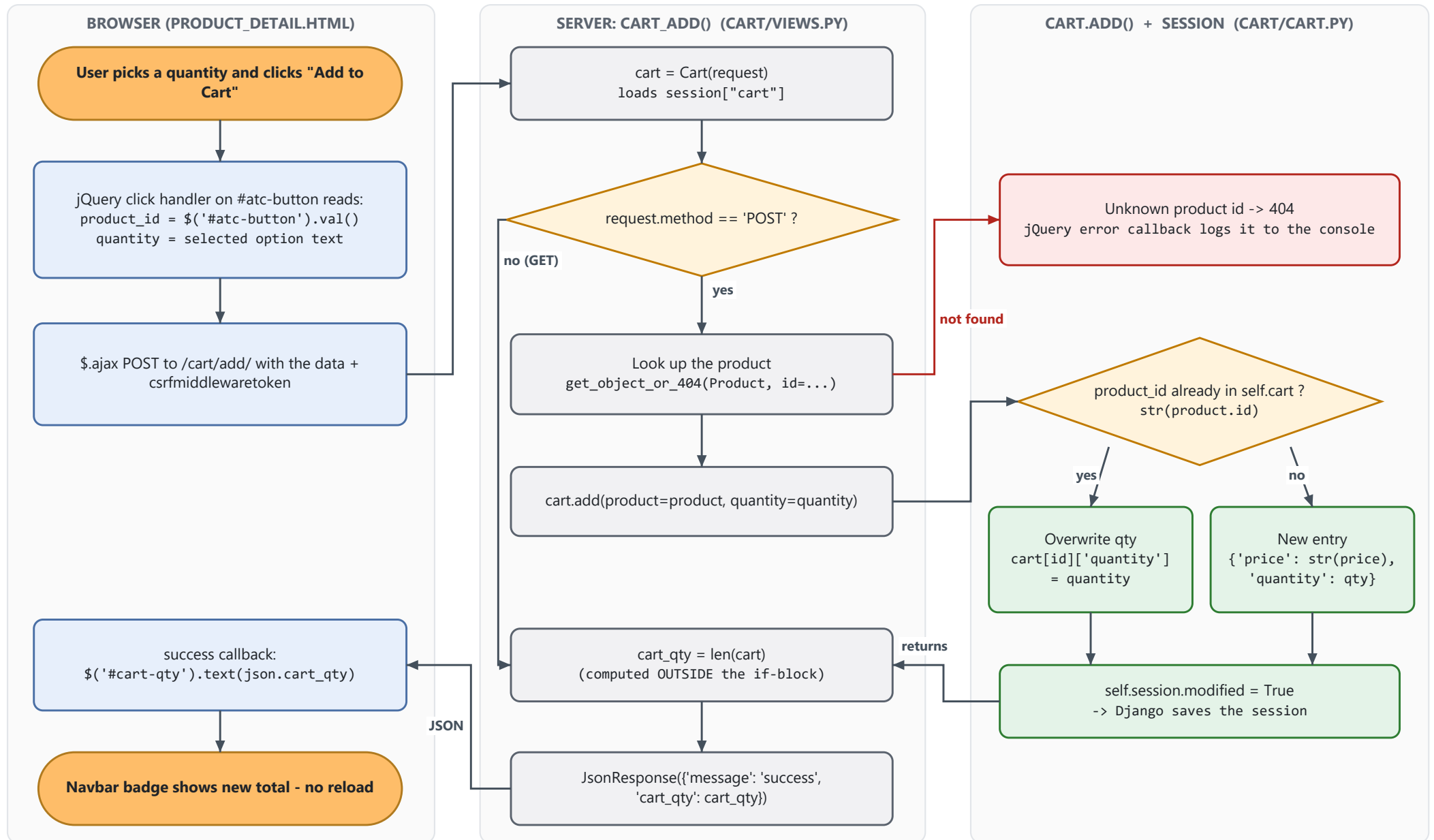


Start / end Browser (template + jQuery) Django view / Python Cart class / session write Decision Error / rejected

len(cart) = SUM of all quantities (not the number of different products). All 3 AJAX views return it as json.cart_qty.

Flow 1 - Add to cart

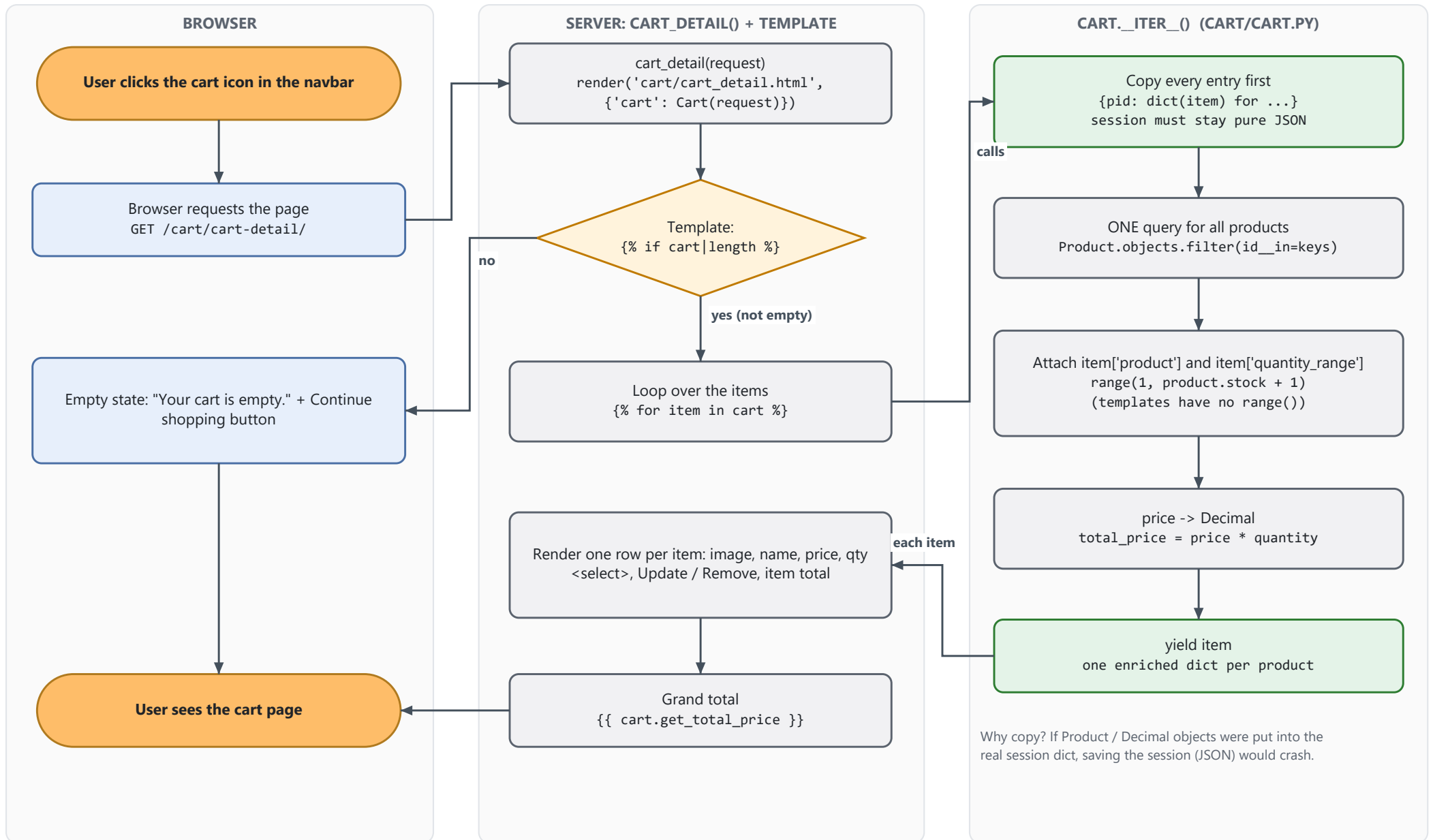
product_detail.html -> POST /cart/add/ -> cart_add() -> Cart.add() -> JSON back -> navbar badge updates without a page reload.



Start / end Browser (template + jQuery) Django view / Python Cart class / session write Decision Error / rejected

Flow 2 - Viewing the cart page

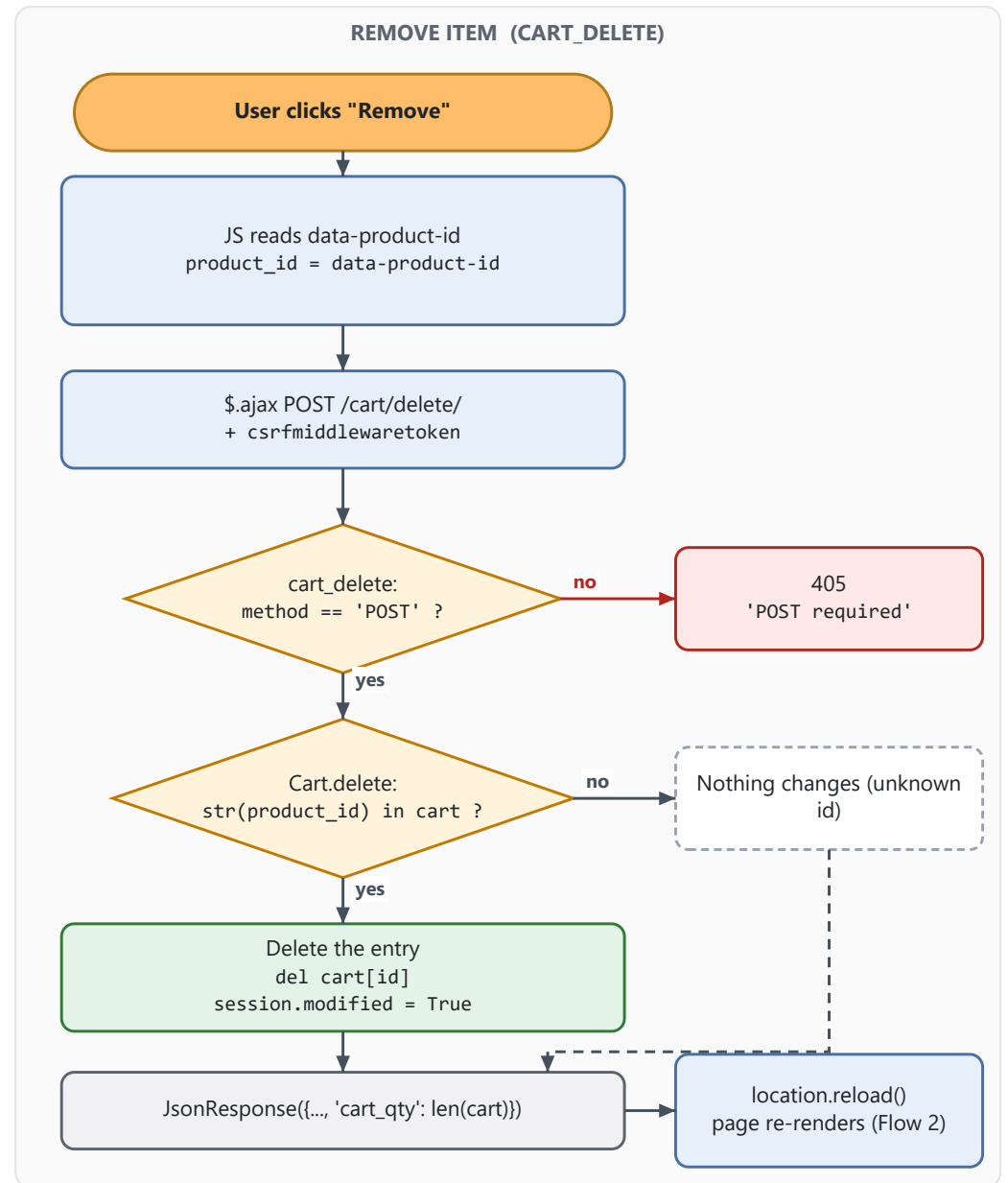
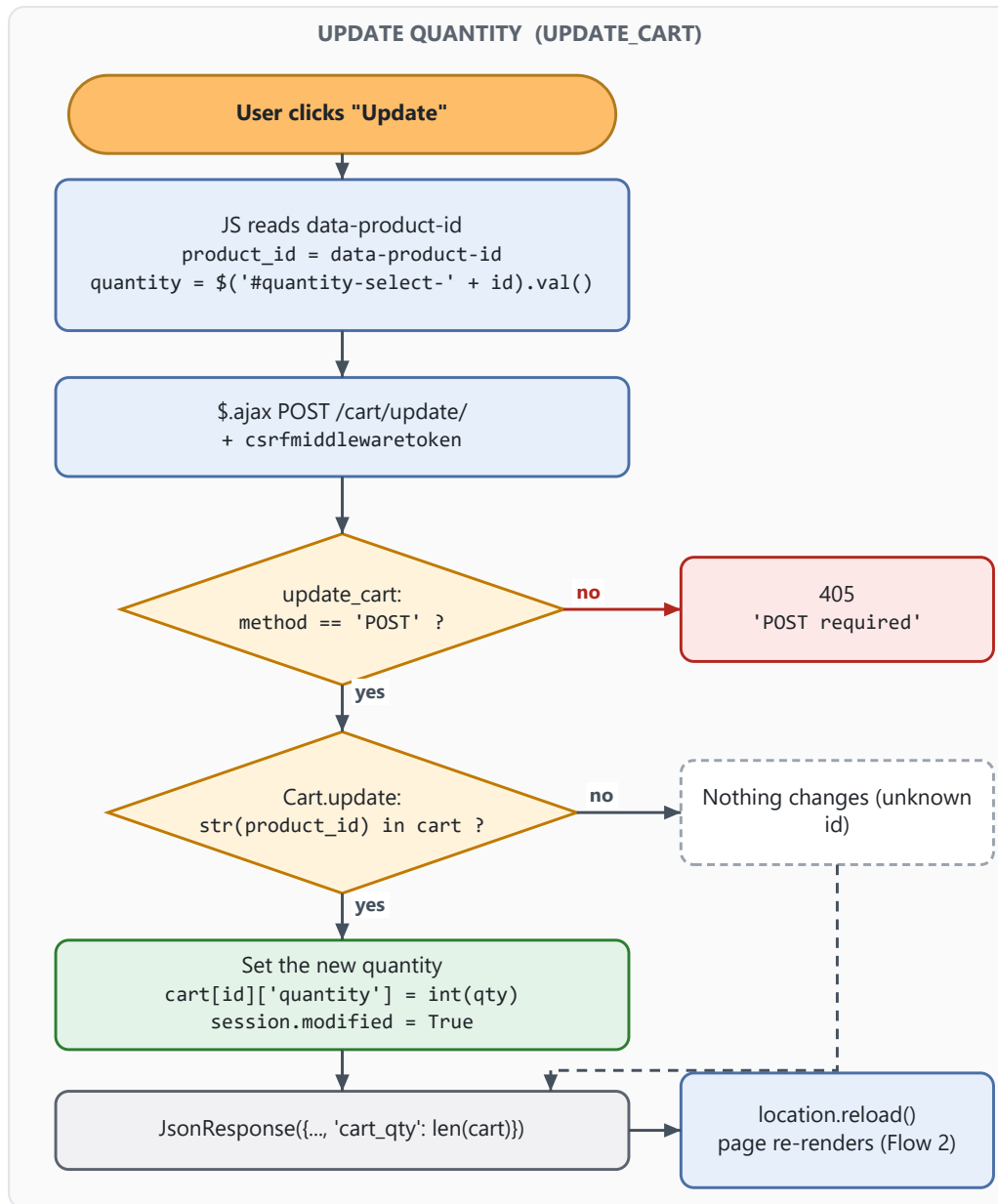
GET /cart/cart-detail/ renders cart_detail.html. Looping over the cart calls Cart.__iter__, which joins the session data with Product rows from the DB.



Start / end Browser (template + jQuery) Django view / Python Cart class / session write Decision Error / rejected

Flow 3 - Updating and removing items

Both buttons on cart_detail.html follow the same pattern: AJAX POST -> view -> Cart method -> JSON -> location.reload() re-renders the page (Flow 2).



Start / end Browser (template + jQuery) Django view / Python Cart class / session write Decision Error / rejected

Cart rules to remember

The non-obvious things we learned while building the cart. Each one caused a real bug or confusion at some point.

1. Session data must be JSON

Only str / int / list / dict can go into request.session. That is why price is stored as str(product.price) and turned back into Decimal only on copies inside `__iter__`.

2. Session keys come back as strings

After a round-trip through JSON, dict keys are always strings. Every Cart method does `product_id = str(product_id)` before looking it up, or "3" and 3 would be different items.

3. Always set session.modified = True

Django only notices assignments to session["cart"] itself. Changing a nested dict (cart[id]["quantity"] = 2) is invisible unless you mark the session as modified.

4. add() overwrites, it does not increase

Adding a product that is already in the cart replaces its quantity with the newly selected one. If "add 2 more" behaviour is wanted later, change it to +=.

5. len(cart) = total quantity

`__len__` sums all quantities, so 2 keyboards + 1 mouse shows "3" in the badge. Both the badge and `{% if cart|length %}` use this.

6. Define variables outside the if-block

`cart_add` used to set `cart_qty` only inside `if request.method == "POST"`. A GET request then hit `UnboundLocalError`. Fixed in commit e967155.

7. Old cart format is cleaned up automatically

Early versions stored non-dict values (e.g. "name", "items") in session["cart"]. `Cart.__init__` now deletes any entry that is not a dict so old sessions do not crash.

8. One DB query per page, not one per item

`__iter__` loads all products with `filter(id__in=...)` instead of calling `Product.objects.get()` inside a loop.

9. Templates have no range()

The quantity dropdown needs 1..stock. `__iter__` attaches `quantity_range = range(1, stock + 1)` so the template can loop over it.

10. Known gap: deleted products

If a product is deleted from the DB while it sits in someone's cart, that item has no "product" key and the cart page will error. Worth handling later.